

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 4: O IMPACTO DO MULTITHREADING EM APLICAÇÕES LIMITADAS POR MEMÓRIA (MEMORY-BOUND) E POR CPU (CPU-BOUND)

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 21/08/2026.

1. EXPERIMENTO

"Implemente dois programas paralelos em C com OpenMP: um limitado por memória, com somas simples em vetores, e outro limitado por CPU, com cálculos matemáticos intensivos. Paralelize com #pragma omp parallel for e meça o tempo de execução variando o número de threads. Análise quando o desempenho melhora, estabiliza ou piora, e reflita sobre como o multithreading de hardware pode ajudar em programas memory-bound, mas atrapalhar em programas compute-bound pela competição por recursos."

2. CÓDIGO

O experimento foi estruturado através do desenvolvimento de dois programas distintos em linguagem C, utilizando a API OpenMP para paralelização via "multithreading". O objetivo é isolar e analisar dois dos principais gargalos de hardware da computação moderna, a largura de banda de memória (Memory-Bound) e a capacidade de processamento da CPU (Compute-Bound).

As execuções foram pensadas para a arquitetura de um processador AMD Ryzen 7 5700X, que possui 8 núcleos físicos, 16 threads lógicas (via SMT) e uma hierarquia de memória cache específica.

Para garantir medições confiáveis, ambos os programas utilizam a função “omp_get_wtime()” para cronometrar a região paralela e exibem os resultados finais na tela, forçando o uso dos dados e impedindo que o compilador elimine os laços de cálculo (otimização de código morto). A paralelização principal em ambos ocorre por meio da diretiva “#pragma omp parallel for”, que divide as iterações dos laços automaticamente entre as threads ativas.

2.1 LIMITAÇÃO NA MEMÓRIA

O primeiro código (limitado_memoria.c) foi projetado para sobrecarregar a comunicação entre o processador e a memória RAM. Para isso, foram alocados três vetores que totalizam cerca de 1,2 GB de dados, um volume que excede propositalmente o Cache L3 da CPU (32 MiB). Isso impede o armazenamento nos caches ultra rápidos, gerando “cache misses” contínuos e forçando o barramento a buscar dados diretamente na RAM.

Como a operação principal é uma soma vetorial muito simples, não há processamento pesado e a CPU passa a maior parte do tempo ociosa, aguardando os dados. Por conta disso, espera-se que o canal de memória sature rapidamente com o aumento das threads. Acima de 4 a 8 threads, o limite físico de transferência é atingido e o uso do Simultaneous Multithreading (SMT) deixa de ser vantajoso. Pelo contrário, o uso de 16 threads competindo pelo mesmo acesso à RAM tende a gerar disputa de recursos e overhead de sincronização, não melhorando, e possivelmente piorando, o tempo de execução.

Figura 1: limitado_memoria.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

// 50 milhões de elementos por vetor (aproximadamente 400 MB por vetor)
#define N 50000000
#define quantidade_repeticoes 10
```

```c
int main() {
    // Alocação dinâmica para evitar estouro de pilha (stack overflow)
    double *A = (double *)malloc(N * sizeof(double));
    double *B = (double *)malloc(N * sizeof(double));
    double *C = (double *)malloc(N * sizeof(double));

    if (A == NULL || B == NULL || C == NULL) {
        fprintf(stderr, "Erro: Falha na alocação de memória.\n");
        return 1;
    }

    // Inicialização simples dos vetores (sem medir tempo)
    for (size_t i = 0; i < N; i++) {
        A[i] = (double)i * 0.1;
        B[i] = (double)i * 0.2;
        C[i] = 0.0;
    }

    int total_threads = 0;
    double tempo_inicio = omp_get_wtime();

    // Região paralela principal
    #pragma omp parallel
    {
        // Pega a quantidade de threads configurada apenas uma vez
        #pragma omp single
        {
            total_threads = omp_get_num_threads();
        }

        // Repito o cálculo para obter uma medição de tempo mais estável e robusta
        for (int r = 0; r < quantidade_repeticoes; r++) {
            #pragma omp for
            for (size_t i = 0; i < N; i++) {
                C[i] = A[i] + B[i];
            }
        }
    }
```

```c
    double tempo_fim = omp_get_wtime();
    double tempo_total = tempo_fim - tempo_inicio;

    // Impede o compilador de otimizar e remover o loop de soma
    // (Consome uma parte de C para garantir que o cálculo foi executado)
    printf("--- Execucao Memory-Bound ---\n");
    printf("Threads ativas:       %d\n", total_threads);
    printf("Tempo total (%d rep):   %f segundos\n", quantidade_repeticoes, tempo_total);
    printf("Tempo por iteracao:    %f segundos\n", tempo_total / quantidade_repeticoes);
    printf("\n");

    // Liberação da memória
    free(A);
    free(B);
    free(C);

    return 0;
}
```

2.2 LIMITAÇÃO NA CPU

O segundo código (limitado_cpu.c) tem o propósito inverso, buscano isolar a CPU da dependência da memória RAM e focar estritamente na capacidade das Unidades de Ponto Flutuante (FPUs). Para eliminar a latência de memória, o programa utiliza um vetor pequeno (cerca de 80 KiB), garantindo que os dados caibam inteiramente nos caches ultrarrápidos locais de cada núcleo (L1 ou L2).

Em vez de focar na movimentação de dados, o laço interno aplica um cálculo matemático extremamente intensivo, repetido 50.000 vezes por elemento com funções pesadas (sin, cos, sqrt, tan), sobrecarregando a lógica aritmética. Como resultado, espera-se uma aceleração quase linear ao escalar a execução de 1 até 8 threads (limite de núcleos físicos). No entanto, ao ativar as 16 threads (SMT), o desempenho deve estagnar: como os cálculos já saturam totalmente as unidades de execução, as threads adicionais alocadas em um mesmo núcleo precisarão compartilhar as mesmas FPUs, sem gerar ganhos no rendimento matemático.

Figura 2: limitado_cpu.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

// N pequeno para caber nos caches locais (L1/L2)
#define N 10000
// M alto para gerar bilhões de operações aritméticas pesadas
#define M 50000

int main() {
    double *A = (double *)malloc(N * sizeof(double));
    if (A == NULL) {
        fprintf(stderr, "Erro: Falha na alocação de memória.\n");
        return 1;
    }

    // Inicialização do vetor (fora da medição)
    for (size_t i = 0; i < N; i++) {
        A[i] = (double)i * 0.001;
    }

    int total_threads = 0;
    double tempo_inicio = omp_get_wtime();

    // Paralelização com OpenMP
    #pragma omp parallel
    {
        #pragma omp single
        {
            total_threads = omp_get_num_threads();
        }

        #pragma omp for schedule(static)
        for (size_t i = 0; i < N; i++) {
            double val = A[i];
            // Laço intensivo de processamento matemático (FPUs)
```

```
        for (int j = 0; j < M; j++) {
            val = sin(val) * cos(val) + sqrt(val + 1.0) - tan(val * 0.001);
        }
        A[i] = val;
    }
}

double tempo_fim = omp_get_wtime();
double tempo_total = tempo_fim - tempo_inicio;

// Impede o compilador de otimizar e descartar as computações matemáticas
printf("--- Execucao Compute-Bound ---\n");
printf("Threads ativas: %d\n", total_threads);
printf("Tempo total de calculo: %f segundos\n", tempo_total);
printf("\n");

free(A);
return 0;
}
```

3. RESULTADOS

O resultado dos testes do programa gerado pelo código do teste “limitado_memoria”, podem ser vistos na Tabela 1 e gráfico da Figura 3. Já os do programa gerado pelo código do programa “limitado_cpu”, podem ser vistos na Tabela 2 e gráfico da Figura 4.

Tabela 1: Testes com limitação por memória

Quantidade de threads	Tempo Memory-Bound (s)
2	0,062539
4	0,060522
6	0,057290
8	0,059164
10	0,058738

12	0,058918
14	0,058714
16	0,061658

Tabela 2: Testes com limitação por CPU

Quantidade de threads	Tempo CPU-Bound (s)
2	4,998030
4	2,548966
6	1,811883
8	1,448879
10	1,214579
12	1,092018
14	0,979387
16	0,995907

Figura 3: Tempo de execução memory bound em função das threads

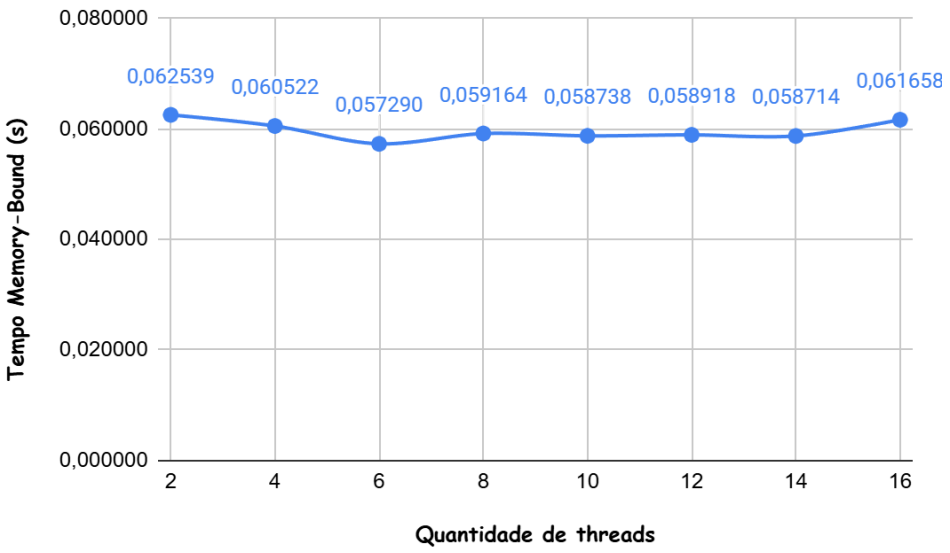
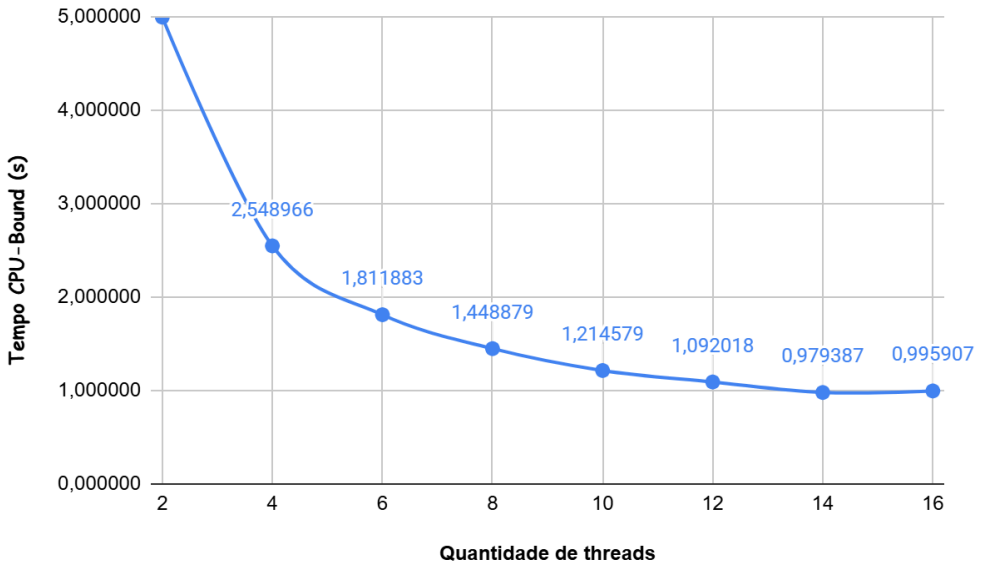


Figura 4: Tempo de execução CPU bound em função das threads



Os dados coletados no experimento confirmam o comportamento teórico esperado para ambos os perfis de carga, evidenciando como a arquitetura do processador reage de maneira distinta à limitação de memória e de capacidade matemática.

A análise da Tabela 1 demonstra que a adição de “threads” tem impacto mínimo no tempo de execução. Com 2“threads”, o programa registrou 0,062539 segundos, atingindo seu pico máximo de eficiência com 6 “threads” (0,057290s). A partir desse ponto, o desempenho entra em platô. Como previsto, o canal de comunicação com a memória RAM saturou rapidamente, tornando-se o gargalo da operação.

Destaca-se ainda que a execução com 16 “threads” (0,061658s) obteve um resultado pior do que com 4 “threads” (0,060522 s). Isso comprova empiricamente que o “Simultaneous Multithreading” (SMT) não beneficia programas Memory-Bound. Pelo contrário, forçar 16 lógicas a disputarem simultaneamente o acesso ao barramento de memória introduziu um custo de contenção de recursos (overhead) que prejudicou o tempo final.

Já na Tabela 2, é ilustrado um cenário de alta escalabilidade inicial, onde o trabalho pesado das Unidades de Ponto Flutuante (FPUs) ditou o ritmo. Ao dobrar o processamento de 2 “threads” (4,998030s) para 4 (2,548966s), o tempo caiu praticamente pela metade, mantendo ganhos agressivos até a marca de 8 *threads*

(1,448879 s). Como os dados cabiam no cache L1/L2 local, a CPU pôde explorar livremente o processamento paralelo em seus 8 núcleos físicos independentes sem depender da RAM.

Entretanto, ao ultrapassar as 8 “threads” e ativar os núcleos lógicos adicionais (SMT), a curva de escalabilidade sofreu uma forte desaceleração. Embora os tempos continuem caindo de forma muito tímida até 14 “threads” (0,979387s), o tempo com a carga máxima de 16 “threads” (0,995907s) regrediu ligeiramente. Esse fenômeno comprova que as FPUs já estavam operando em sua capacidade máxima. Forçar o compartilhamento da mesma unidade de cálculo entre duas “threads” alocadas no mesmo núcleo gerou uma competição por recursos que fez o desempenho estagnar.

4. CONSIDERAÇÕES FINAIS

O experimento demonstrou que aumentar o número de *threads* não garante ganhos automáticos de desempenho. O sucesso da paralelização depende diretamente do tipo de gargalo do sistema.

Em teoria, o “multithreading” de hardware é útil para programas limitados por memória, pois permite que o processador execute outra tarefa enquanto aguarda a chegada dos dados. Contudo, o experimento provou que esse benefício acaba assim que o canal de comunicação com a memória atinge sua capacidade máxima. A partir desse ponto, forçar o uso de 16 “threads” gera apenas competição por recursos e piora o tempo final.

Já nos programas focados em cálculos intensivos, o desempenho escala de forma eficiente até o limite de núcleos físicos do processador (8 threads). Ultrapassar essa marca também traz ganhos mínimos, pois as unidades matemáticas já estão operando em sua capacidade máxima e precisam ser compartilhadas entre as tarefas.

Conclui-se que a paralelização eficiente exige conhecer a fundo a arquitetura da máquina. É fundamental dimensionar as tarefas de acordo com os núcleos reais disponíveis, equilibrando o uso do processador com os limites físicos de transferência de dados..